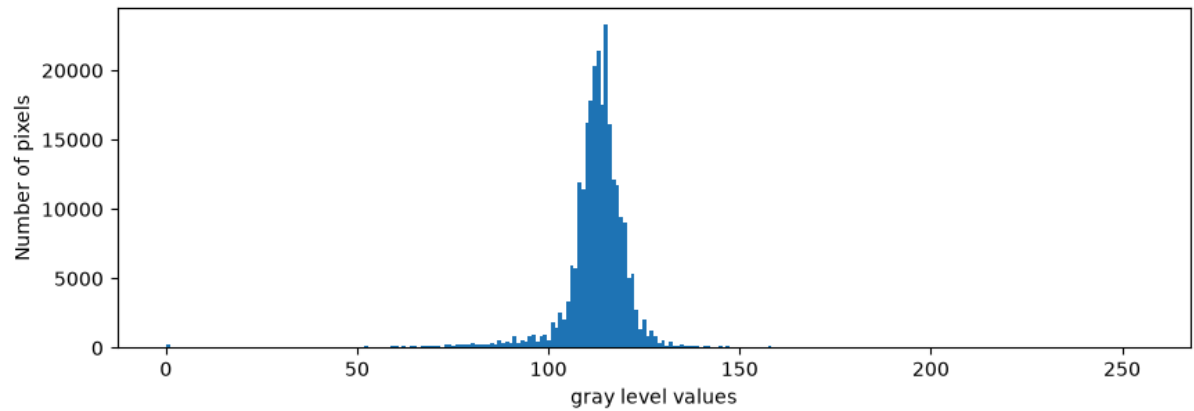
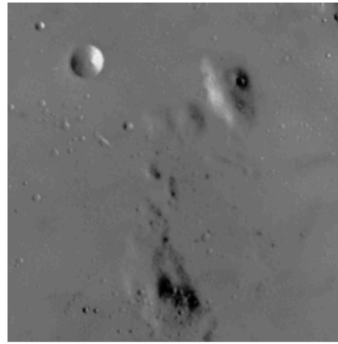


Ali Hisham AlJamed 2240005868 7MA1/MA01

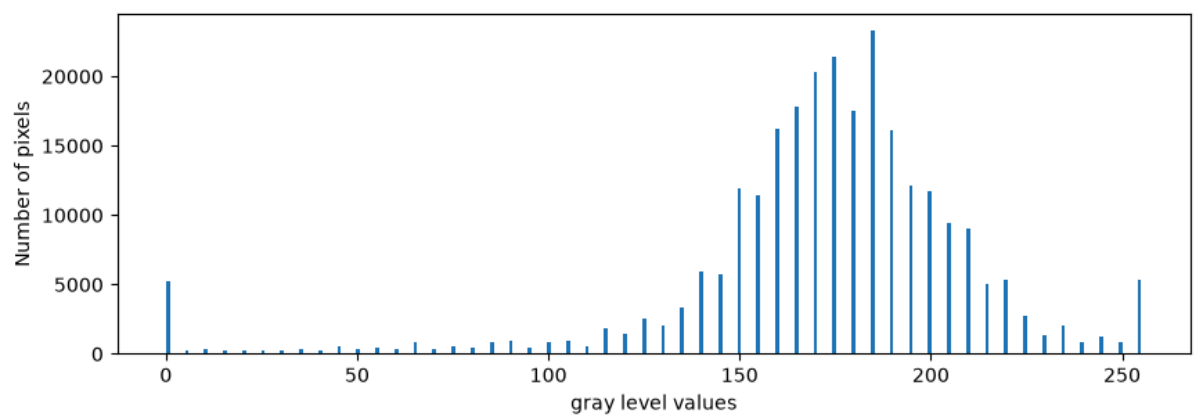
```
In [ ]: import cv2
img = cv2.imread('IAU_Logo.png', cv2.IMREAD_GRAYSCALE)
# Set the threshold values
threshold_values = [0, 50, 100, 150, 200]
# Apply thresholding with different threshold values
for threshold_value in threshold_values:
    ret, thresh = cv2.threshold(img, threshold_value, 255, cv2.THRESH_BINARY)
    cv2.imshow(f'Thresholded Image ({threshold_value})', thresh)
# Display the original image
cv2.imshow('Original Image', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
In [1]: import matplotlib
import matplotlib.pyplot as plt
import numpy as np
from skimage import data, img_as_float
from skimage import exposure
img = data.moon()
# Contrast stretching
p2, p98 = np.percentile(img, (2, 98))
img_rescale = exposure.rescale_intensity(img, in_range=(p2, p98))
fig = plt.figure(figsize=(10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(img, cmap = 'gray')
plt.axis('off')
plt.title('Original image')
fig.add_subplot(2, 1, 2)
plt.hist(img.flat, bins = 256, range=(0, 255))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.show()
fig = plt.figure(figsize=(10, 7))
fig.add_subplot(2, 1, 1)
plt.imshow(img_rescale, cmap = 'gray')
plt.axis('off')
plt.title('Contrast stretch')
fig.add_subplot(2, 1, 2)
plt.hist(img_rescale.flat, bins = 256, range=(0, 255))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.show()
```

Original image



Contrast stretch



Assesment:

Task 1:

```

In [2]: import matplotlib
import matplotlib.pyplot as plt
import numpy as np
from skimage import data, img_as_float
from skimage import exposure

img = data.moon()

# Rescale intensity using the 3rd and 80th percentiles
p3, p80 = np.percentile(img, (3, 80))
img_rescale = exposure.rescale_intensity(img, in_range=(p3, p80))

fig = plt.figure(figsize=(10, 7))

fig.add_subplot(2, 1, 1)
plt.imshow(img_rescale, cmap='gray')
plt.axis('off')
plt.title('Rescaled image (3rd - 80th percentiles)')

fig.add_subplot(2, 1, 2)
plt.hist(img_rescale.flat, bins=256, range=(0, 255))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.title('Histogram after rescaling')

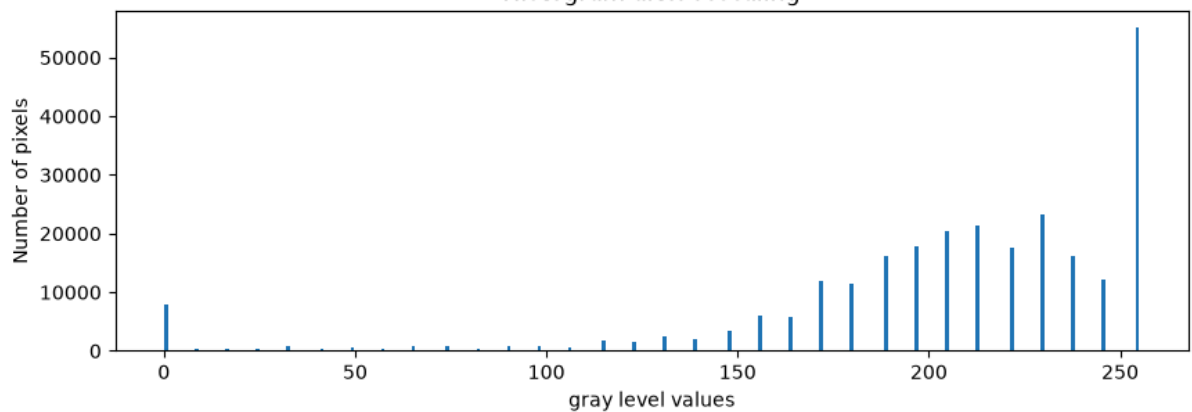
plt.show()

```

Rescaled image (3rd - 80th percentiles)



Histogram after rescaling



Task 2:

```
In [3]: # Histogram equalization
img_equalized = exposure.equalize_hist(img)

fig = plt.figure(figsize=(10, 7))

fig.add_subplot(2, 1, 1)
plt.imshow(img_equalized, cmap='gray')
plt.axis('off')
plt.title('Histogram Equalized Image')

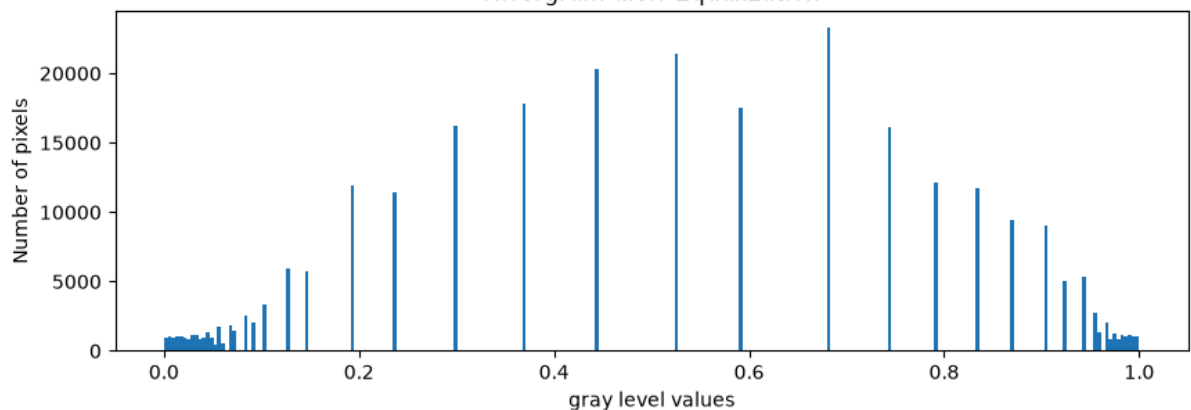
fig.add_subplot(2, 1, 2)
plt.hist(img_equalized.flat, bins=256, range=(0, 1))
plt.xlabel('gray level values')
plt.ylabel('Number of pixels')
plt.title('Histogram after Equalization')

plt.show()
```

Histogram Equalized Image



Histogram after Equalization



Task 3:

```
In [4]: # Load images
reference = data.rocket()
image = data.chelsea()

# Histogram matching
matched = exposure.match_histograms(image, reference, channel_axis=-1)

fig = plt.figure(figsize=(12, 8))
```

```
# Chelsea original
fig.add_subplot(2, 3, 1)
plt.imshow(image)
plt.axis('off')
plt.title('Chelsea')

# Rocket reference
fig.add_subplot(2, 3, 2)
plt.imshow(reference)
plt.axis('off')
plt.title('Rocket (Reference)')

# Matched image
fig.add_subplot(2, 3, 3)
plt.imshow(matched)
plt.axis('off')
plt.title('Histogram Matched')

# Chelsea histogram
fig.add_subplot(2, 3, 4)
plt.hist(image.ravel(), bins=256)
plt.title('Chelsea Histogram')

# Rocket histogram
fig.add_subplot(2, 3, 5)
plt.hist(reference.ravel(), bins=256)
plt.title('Rocket Histogram')

# Matched histogram
fig.add_subplot(2, 3, 6)
plt.hist(matched.ravel(), bins=256)
plt.title('Matched Histogram')

plt.tight_layout()
plt.show()
```

Chelsea



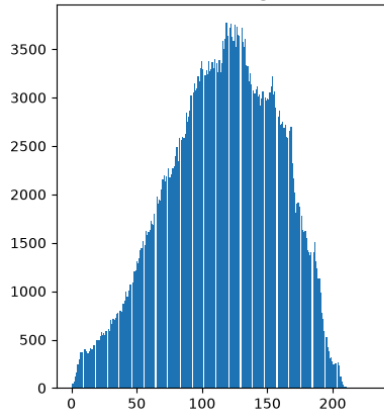
Rocket (Reference)



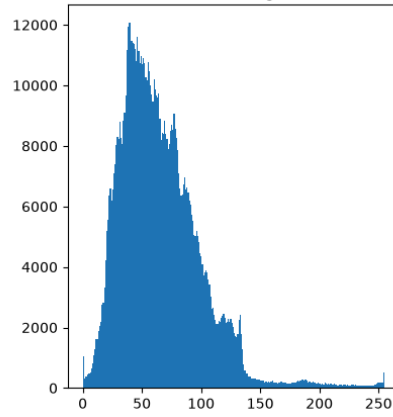
Histogram Matched



Chelsea Histogram



Rocket Histogram



Matched Histogram

